

PEP 3107 – Function Annotations

Author: Collin Winter <[collinwinter at google.com](mailto:collinwinter@google.com)>, Tony Lownds <[tony at lownds.com](mailto:tony@lownds.com)>

Status: Final

Type: Standards Track

Created: 02-Dec-2006

Python-Version: 3.0

Post-History:

Source:

<https://github.com/python/peps/blob/main/peps/pep-3107.rst>

Abstract (#abstract)

This PEP introduces a syntax for adding arbitrary metadata annotations to Python functions [1] (#functerm).

Rationale (#rationale)

Because Python's 2.x series lacks a standard way of annotating a function's parameters and return values, a variety of tools and libraries have appeared to fill this gap. Some utilise the decorators introduced in PEP 318 (../pep-0318/), while others parse a function's docstring, looking for annotations there.

This PEP aims to provide a single, standard way of specifying this information, reducing the confusion caused by the wide variation in mechanism and syntax that has existed until this point.

Fundamentals of Function Annotations (#fundamentals-of-function-annotations)

Before launching into a discussion of the precise ins and outs of Python 3.0's function annotations, let's first talk broadly about what annotations are and are not:

1. Function annotations, both for parameters and return values, are completely optional.

Last

modified

2025-02-

01

08:59:27

UTC

- Function annotations are nothing more than a way of associating arbitrary Python expressions with various parts of a function at compile-time.

By itself, Python does not attach any particular meaning or significance to annotations. Left to its own, Python simply makes these expressions available as described in [Accessing Function Annotations](#) ([#accessing-function-annotations](#)) below.

The only way that annotations take on meaning is when they are interpreted by third-party libraries. These annotation consumers can do anything they want with a function's annotations. For example, one library might use string-based annotations to provide improved help messages, like so:

```
def compile(source: "something compilable",
            filename: "where the compilable thing comes from",
            mode: "is this a single statement or a suite?"):
    ...
```

Another library might be used to provide typechecking for Python functions and methods. This library could use annotations to indicate the function's expected input and return types, possibly something like:

```
def haul(item: Haulable, *vargs: PackAnimal) -> Distance:
    ...
```

However, neither the strings in the first example nor the type information in the second example have any meaning on their own; meaning comes from third-party libraries alone.

- Following from point 2, this PEP makes no attempt to introduce any kind of standard semantics, even for the built-in types. This work will be left to third-party libraries.

Syntax [\(#syntax\)](#)

Parameters [\(#parameters\)](#)

Annotations for parameters take the form of optional expressions that follow the parameter name:

```
def foo(a: expression, b: expression = 5):
    ...
```

In pseudo-grammar, parameters now look like `identifier [: expression] [= expression]`. That is, annotations always precede a parameter's default value and both annotations and default values are optional. Just like how equal signs are used to indicate a default value, colons are used to mark annotations. All annotation expressions are evaluated when the function definition is executed, just like default values.

Annotations for excess parameters (i.e., `*args` and `**kwargs`) are indicated similarly:

```
def foo(*args: expression, **kwargs: expression):
    ...
```

Annotations for nested parameters always follow the name of the parameter, not the last parenthesis. Annotating all parameters of a nested parameter is not required:

```
def foo((x1, y1: expression),
        (x2: expression, y2: expression)=(None, None)):
    ...
```

Return Values (#return-values)

The examples thus far have omitted examples of how to annotate the type of a function's return value. This is done like so:

```
def sum() -> expression:
    ...
```

That is, the parameter list can now be followed by a literal `->` and a Python expression. Like the annotations for parameters, this expression will be evaluated when the function definition is executed.

The grammar for function definitions [11] (#grammar) is now:

```

decorator: '@' dotted_name [ '(' [arglist] ')' ] NEWLINE
decorators: decorator+
funcdef: [decorators] 'def' NAME parameters ['->' test] ':' suite
parameters: '(' [typedargslist] ')'
typedargslist: ((tfpdef ['=' test] ',')*
                ('*' [tname] (',' tname ['=' test])* [',' '**' tname]
                | '**' tname)
                | tfpdef ['=' test] (',' tfpdef ['=' test])* [',''])
tname: NAME [':' test]
tfpdef: tname | '(' tfplist ')'
tfplist: tfpdef (',' tfpdef)* [',']

```

`.lambda` (#lambda)

`lambda`'s syntax does not support annotations. The syntax of `lambda` could be changed to support annotations, by requiring parentheses around the parameter list. However it was decided [12] (#id25) not to make this change because:

1. It would be an incompatible change.
2. Lambdas are neutered anyway.
3. The lambda can always be changed to a function.

Accessing Function Annotations (#accessing-function-annotations)

Once compiled, a function's annotations are available via the function's `__annotations__` attribute. This attribute is a mutable dictionary, mapping parameter names to an object representing the evaluated annotation expression

There is a special key in the `__annotations__` mapping, "return". This key is present only if an annotation was supplied for the function's return value.

For example, the following annotation:

```
def foo(a: 'x', b: 5 + 6, c: list) -> max(2, 9):
    ...
```

would result in an `__annotations__` mapping of

```
{
    'a': 'x',
    'b': 11,
    'c': list,
    'return': 9
}
```

The `return` key was chosen because it cannot conflict with the name of a parameter; any attempt to use `return` as a parameter name would result in a `SyntaxError`.

`__annotations__` is an empty, mutable dictionary if there are no annotations on the function or if the function was created from a `lambda` expression.

Use Cases (#use-cases)

In the course of discussing annotations, a number of use-cases have been raised. Some of these are presented here, grouped by what kind of information they convey. Also included are examples of existing products and packages that could make use of annotations.

- Providing typing information
 - Type checking ([3] (#typecheck), [4] (#maxime))
 - Let IDEs show what types a function expects and returns ([16] (#idle))
 - Function overloading / generic functions ([21] (#scaling))
 - Foreign-language bridges ([17] (#jython), [18] (#ironpython))
 - Adaptation ([20] (#adaptationpost), [19] (#pyprotocols))
 - Predicate logic functions
 - Database query mapping
 - RPC parameter marshaling ([22] (#rpyc))
- Other information
 - Documentation for parameters and return values ([23] (#pydoc))

Standard Library (#standard-library)

`pydoc` and `inspect` (#pydoc-and-inspect)

The `pydoc` module should display the function annotations when displaying help for a function. The `inspect` module should change to support annotations.

Relation to Other PEPs (#relation-to-other-peps)

Function Signature Objects (PEP 362) (#function-signature-objects-pep-362)

Function Signature Objects should expose the function’s annotations. The `Parameter` object may change or other changes may be warranted.

Implementation (#implementation)

A reference implementation has been checked into the py3k (formerly “p3yk”) branch as revision 53170 [10] (#id24).

Rejected Proposals (#rejected-proposals)

- The BDFL rejected the author’s idea for a special syntax for adding annotations to generators as being “too ugly” [2] (#rejectgensyn).
- Though discussed early on ([5] (#threadgen), [6] (#threadhof)), including special objects in the stdlib for annotating generator functions and higher-order functions was ultimately rejected as being more appropriate for third-party libraries; including them in the standard library raised too many thorny issues.
- Despite considerable discussion about a standard type parameterisation syntax, it was decided that this should also be left to third-party libraries. ([7] (#threadimmlist), [8] (#threadmixing), [9] (#emphasistpls)).
- Despite yet more discussion, it was decided not to standardize a mechanism for annotation interoperability. Standardizing interoperability conventions at this point would be premature. We would rather let these conventions develop organically, based on real-world usage and necessity, than try to force all users into some contrived scheme. ([13] (#interop0), [14] (#interop1), [15] (#interop2)).

References and Footnotes (#references-and-footnotes)

[1] (#id1)

Unless specifically stated, “function” is generally used as a synonym for “callable” throughout this document.

[2] (#id15)

<https://mail.python.org/pipermail/python-3000/2006-May/002103.html>

(<https://mail.python.org/pipermail/python-3000/2006-May/002103.html>)

3 (#id4)]

<http://web.archive.org/web/20070730120117/http://oakwinter.com/code/typecheck/>

(<http://web.archive.org/web/20070730120117/http://oakwinter.com/code/typecheck/>)

4 (#id5)]

<http://web.archive.org/web/20070603221429/http://maxrepo.info/>

(<http://web.archive.org/web/20070603221429/http://maxrepo.info/>)

5 (#id16)]

<https://mail.python.org/pipermail/python-3000/2006-May/002091.html>

(<https://mail.python.org/pipermail/python-3000/2006-May/002091.html>)

6 (#id17)]

<https://mail.python.org/pipermail/python-3000/2006-May/001972.html>

(<https://mail.python.org/pipermail/python-3000/2006-May/001972.html>)

7 (#id18)]

<https://mail.python.org/pipermail/python-3000/2006-May/002105.html>

(<https://mail.python.org/pipermail/python-3000/2006-May/002105.html>)

8 (#id19)]

<https://mail.python.org/pipermail/python-3000/2006-May/002209.html>

(<https://mail.python.org/pipermail/python-3000/2006-May/002209.html>)

9 (#id20)]

<https://mail.python.org/pipermail/python-3000/2006-June/002438.html>

(<https://mail.python.org/pipermail/python-3000/2006-June/002438.html>)

10 (#id14)]

<http://svn.python.org/view?rev=53170&view=rev> (<http://svn.python.org/view?rev=53170&view=rev>)

11 (#id2)]

http://docs.python.org/reference/compound_stmts.html#function-definitions (http://docs.python.org/reference/compound_stmts.html#function-definitions)

12 (#id3)]

<https://mail.python.org/pipermail/python-3000/2006-May/001613.html>

(<https://mail.python.org/pipermail/python-3000/2006-May/001613.html>)

13 (#id21)]

<https://mail.python.org/pipermail/python-3000/2006-August/002895.html> (<https://mail.python.org/pipermail/python-3000/2006-August/002895.html>)

[August/002895.html](#))

[14 (#id22)]

<https://mail.python.org/pipermail/python-ideas/2007-January/000032.html> (<https://mail.python.org/pipermail/python-ideas/2007-January/000032.html>)

[15 (#id23)]

<https://mail.python.org/pipermail/python-list/2006-December/420645.html> (<https://mail.python.org/pipermail/python-list/2006-December/420645.html>)

[16 (#id6)]

<http://www.python.org/idle/doc/idle2.html#Tips>
(<http://www.python.org/idle/doc/idle2.html#Tips>)

[17 (#id8)]

<http://www.jython.org/Project/index.html>
(<http://www.jython.org/Project/index.html>)

[18 (#id9)]

<http://www.codeplex.com/Wiki/View.aspx?ProjectName=IronPython>
(<http://www.codeplex.com/Wiki/View.aspx?ProjectName=IronPython>)

[19 (#id11)]

<http://peak.telecommunity.com/PyProtocols.html>
(<http://peak.telecommunity.com/PyProtocols.html>)

[20 (#id10)]

<http://www.artima.com/weblogs/viewpost.jsp?thread=155123>
(<http://www.artima.com/weblogs/viewpost.jsp?thread=155123>)

[21 (#id7)]

<http://www-128.ibm.com/developerworks/library/l-cppeak2/> (<http://www-128.ibm.com/developerworks/library/l-cppeak2/>)

[22 (#id12)]

<http://rpyc.wikispaces.com/> (<http://rpyc.wikispaces.com/>)

[23 (#id13)]

<http://docs.python.org/library/pydoc.html>
(<http://docs.python.org/library/pydoc.html>)

Copyright (#copyright)

This document has been placed in the public domain.